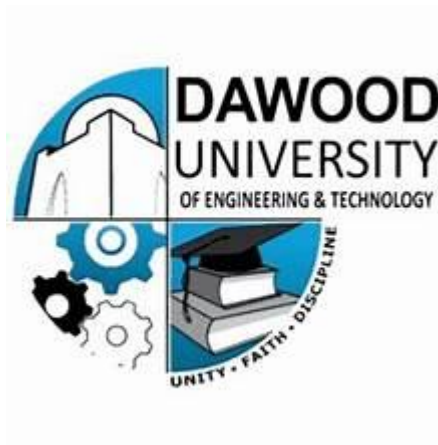




Project Name: Playing Card Detection

Group Members: 73, 93, 33, 65

**Dawood University of Engineering &
Technology**

Playing Card Detection System - Project Documentation

Executive Summary

This project implements a real-time playing card detection system using YOLOv8 deep learning architecture. The system detects and classifies all 52 playing cards from a standard deck through a live webcam feed, providing accurate identification with bounding boxes and confidence scores.

1. Introduction

1.1 Project Overview

The Playing Card Detection System is an AI-powered computer vision application that recognizes playing cards in real-time. This technology has applications in automated card games, casino monitoring, card counting prevention, and interactive gaming experiences.

1.2 Objectives

- Train a deep learning model to accurately detect all 52 playing cards
- Deploy through a browser-based interface using ONNX Runtime Web
- Run entirely locally without requiring cloud services
- Achieve real-time detection with confidence scores displayed per card

1.3 Key Features

- Real-time detection through webcam
- Recognition of all 52 standard playing cards
- High accuracy with confidence scores
- Browser-based interface — no installation required
- Local processing for privacy and speed

2. Technology Stack

2.1 Training Environment

- Platform: Google Colab
- GPU: Tesla T4 (Cloud GPU)
- Framework: Ultralytics YOLOv8
- Language: Python 3.10+

2.2 Deployment Environment

- Runtime: ONNX Runtime Web
- Interface: HTML5, JavaScript, Canvas API
- Browser: Chrome/Edge (WASM support required)
- Model Format: ONNX (Open Neural Network Exchange, opset 12)

2.3 Key Libraries

Component	Technology
Training Platform	Google Colab (Tesla T4 GPU)
Detection Framework	Ultralytics YOLOv8
Language	Python 3.10+
Deployment Runtime	ONNX Runtime Web
Interface	HTML5, JavaScript, Canvas API
Dataset Management	Roboflow
Model Format	ONNX (opset 12)

3. Dataset

3.1 Dataset Source

- Provider: Roboflow Community
- Workspace: poker-eqo1i
- Project: playing-card-dedbm
- Total Classes: 52 (all standard playing cards)

3.2 Card Classes

Suit	Cards
Clubs (C)	AC, 2C, 3C, 4C, 5C, 6C, 7C, 8C, 9C, 10C, JC, QC, KC
Diamonds (D)	AD, 2D, 3D, 4D, 5D, 6D, 7D, 8D, 9D, 10D, JD, QD, KD
Hearts (H)	AH, 2H, 3H, 4H, 5H, 6H, 7H, 8H, 9H, 10H, JH, QH, KH
Spades (S)	AS, 2S, 3S, 4S, 5S, 6S, 7S, 8S, 9S, 10S, JS, QS, KS

3.3 Data Split

- Training Set: ~70%
- Validation Set: ~20%
- Test Set: ~10%

3.4 Data Augmentation Applied

- Horizontal flipping (50% probability)
- Rotation (± 10 degrees)
- Translation ($\pm 10\%$), Scale variation (0.5x to 1.5x)
- HSV color jittering (hue, saturation, value)
- Mosaic augmentation
- Mixup augmentation (10% probability)

4. Methodology

4.1 YOLOv8 Architecture

YOLOv8 (You Only Look Once version 8) is a state-of-the-art object detection model that processes images in a single forward pass, divides images into grids to predict bounding boxes, uses anchor-free detection for better performance, and employs CSPDarknet backbone for feature extraction.

4.2 Model Selection — YOLOv8 Small (yolov8s)

- Good balance between accuracy and speed
- ~11 million parameters
- Input Shape: (1, 3, 640, 640)
- Output Shape: (1, 56, 8400)
- Backbone: CSPDarknet with C2f blocks
- Detection Head: Anchor-free Detect head (52 classes)

4.3 Training Configuration

Hyperparameter	Value
Epochs	200 (Early stopped at 192)
Batch Size	16
Image Size	640x640 pixels
Optimizer	AdamW
Initial Learning Rate	0.001
Final Learning Rate	0.00001
Weight Decay	0.0005
Warmup Epochs	3
Patience (Early Stop)	30
Box Loss Gain	7.5
Class Loss Gain	0.5
DFL Loss Gain	1.5

5. Model Performance & Results

5.1 Final Validation Metrics (Best Model — Epoch 162)

Metric	Value
mAP50	99.4%
mAP50-95	90.1%
Precision	96.9%
Recall	98.0%

5.2 Training Progress Highlights

Epoch	mAP50	mAP50-95
-------	-------	----------

1	1.05%	0.68%
20	68.4%	47.5%
50	98.5%	78.3%
100	99.1%	86.5%
162 (Best)	99.4%	90.0%

5.3 Inference Speed

Note: Training inference was measured on GPU (Google Colab Tesla T4). Browser-based inference runs on CPU via ONNX Runtime Web — speed varies by device.

Environment	Inference Time	FPS
Google Colab (Tesla T4 GPU)	~5ms	30+
Browser (CPU / WASM)	~800–2800ms	1–5

6. System Architecture

6.1 Training Pipeline

Roboflow Dataset → Google Colab → YOLOv8 Training → PyTorch Model (.pt) → ONNX Export (.onnx) → Download

6.2 Deployment Pipeline

User Browser → Load ONNX Model → Access Webcam → Capture Frame → Preprocess (640x640, normalize) → Model Inference → NMS Post-processing → Draw Bounding Boxes → Display on Canvas

6.3 Web Interface Components

- Model Loader: Loads ONNX model into browser memory via file upload
- Camera Handler: Accesses and manages webcam stream
- Preprocessing Module: Converts video frames to 640x640 normalized tensor
- Inference Engine: Runs model predictions using ONNX Runtime Web (WASM)
- Post-processor: Applies confidence threshold (0.35) and NMS filtering
- Visualization Layer: Draws bounding boxes and labels on HTML5 canvas
- Statistics Monitor: Displays real-time FPS, detection count, and inference time

7. Implementation Details

7.1 Image Preprocessing Steps

1. Capture video frame from webcam
2. Resize to 640x640 pixels
3. Normalize pixel values (0-255 to 0-1)
4. Convert to RGB format
5. Reshape to [1, 3, 640, 640] tensor
6. Feed to ONNX model

7.2 Post-processing

1. Extract model output (bounding boxes + class scores)
2. Apply confidence threshold (>0.35) — lowered for better detection
3. Perform Non-Maximum Suppression (NMS, IoU threshold 0.45)
4. Scale boxes back to original frame size
5. Map class indices to card names (e.g., QH = Queen of Hearts)
6. Draw results on canvas overlay

8. Challenges and Solutions

8.1 Challenge: Browser Performance

Issue: ONNX Runtime Web runs on CPU only — no GPU access in browser.

Solution: Used WASM execution provider, optimized preprocessing, async frame processing.

8.2 Challenge: Overlapping Cards

Issue: Cards held in fan-shape have partially hidden faces.

Solution: Model detects visible corners/faces. Best results with cards shown individually.

8.3 Challenge: Lighting Variations

Issue: Cards appear different under various lighting conditions.

Solution: Applied extensive HSV color jittering during training augmentation.

8.4 Challenge: Model Size vs Speed

Issue: Larger models are more accurate but slower in browser.

Solution: Used YOLOv8s as optimal balance with configurable confidence threshold.

9. How to Run

Step 1 — Train the Model (Google Colab)

1. Open Card_Detection_Model_1.ipynb in Google Colab
2. Run all cells to train the model
3. Download: runs/detect/card_detector_v2/weights/best.onnx

Step 2 — Run the Web Interface

1. Save playing_card_detection.html to your local machine
2. Place best.onnx in the SAME folder as the HTML file
3. Open the HTML file in Chrome or Edge
4. Click 'Choose Model File' and select best.onnx
5. Wait for green message: Model loaded! Ready to detect cards
6. Allow webcam access when browser asks
7. Click Start Detection

10. Troubleshooting

Problem	Solution
Model wont load	Make sure best.onnx is selected via Choose Model File button
All stats showing 0	Upload ONNX file first, THEN click Start Detection
Low FPS in browser	Normal — browser runs on CPU only, not GPU
Camera not working	Allow camera permission in Chrome/Edge settings
Card not detected	Hold card flat, ensure good lighting, use Chrome
Detection wrong label	Model confidence below threshold — improve lighting

11. Project Structure

File	Description
Card_Detection_Model_1.ipynb	Training notebook (Google Colab)
playing_card_detection.html	Browser-based detection interface
best.onnx	Exported trained model (ONNX format)
Playing_Card_Detection_Project_Report.pdf	This project report
README.md	Project documentation for GitHub

12. Real-World Applications

- Casino Monitoring — Automated card tracking and fraud detection
- Online Gaming — Enhanced AR card games and virtual poker
- Education — Interactive learning tools for card games
- Accessibility — Assistive technology for visually impaired players
- Card Counting Prevention — Real-time detection in casinos
- Mobile Gaming — Integration with mobile card game apps

13. Conclusion

This Playing Card Detection System demonstrates the practical application of modern deep learning techniques in creating accessible, real-time computer vision applications. By combining YOLOv8's powerful object detection capabilities with browser-based ONNX Runtime deployment, we created a system that is both highly accurate (mAP50: 99.4%) and user-friendly.

The project successfully bridges the gap between complex AI models and everyday users, proving that sophisticated machine learning applications can run efficiently in web browsers without requiring specialized hardware or technical expertise.

Key Takeaways

- YOLOv8 provides excellent performance for real-time object detection (99.4% mAP50)
- Browser-based deployment makes AI accessible to everyone
- Proper data augmentation is crucial for robust model performance
- ONNX format enables cross-platform model deployment
- Local processing ensures privacy and eliminates cloud dependency

14. References

1. Ultralytics YOLOv8 Documentation: <https://docs.ultralytics.com/>
2. Roboflow Dataset Platform: <https://roboflow.com/>
3. ONNX Runtime Documentation: <https://onnxruntime.ai/>
4. Redmon, J. et al. (2016). You Only Look Once: Unified, Real-Time Object Detection.
5. Computer Vision Best Practices — Various academic papers and tutorials

Group Members — BS Artificial Intelligence, DUET Karachi

Roll Numbers: 73 | 93 (Hamza Asif) | 33 | 65

Subject: Machine Learning | Instructor: Sir Hamza Farooqi